

Docker Compose

for Devs

From concept to practice with analogies from our world

From one to many containers

3 Hands-on Exercises

Cheat Sheet

Table of Contents

01	What is Docker Compose?	2
02	Pre-requisite: remember Docker	3
03	What is Docker Compose used for?	4
04	Installing Docker Compose	5
05	Essential Concepts	6
06	Exercise 1. Starting your first Compose	8
07	Exercise 2. App with database	9
08	Exercise 3. Complete dev environment	11
09	Cheat Sheet	13
10	Next Steps	14

How to use this mini book

Read from beginning to end the first time.

The chapters were designed in sequence: each one sets the stage for the next. If you jump straight to the exercises without reading the concepts, you might get lost with some commands.

Come back whenever you need to.

The Cheat Sheet was made to stay open while you practice. Keep it on your second monitor or printed next to your computer.

Actually do the exercises.

Reading about Docker teaches the theory. Running the commands is what makes it stick. Open the terminal and execute each step. There are no shortcuts here.

01 • What is Docker Compose?

You've already learned how to start a container with Docker. But what about when your application needs more than one? An API, a database, a cache service. Managing all of that by hand, container by container, becomes a nightmare very quickly. That's where Docker Compose comes in.

Docker Compose is a tool that lets you define and run applications with **multiple containers** using a single configuration file: **docker-compose.yml**. With one command you bring everything up. With another you tear it all down. Simple as that.

>> Imagine you are organizing a themed dinner party for your friends. Instead of calling each vendor separately, the baker, the florist, the DJ, you hire an event coordinator who handles everything at once. You just say what you want in your event script and they make sure each piece is in the right place at the right time. The docker-compose.yml is that script and Docker Compose is the coordinator.

[DICA] Do I need to know Docker first?

Yes! Docker Compose is built on top of Docker. If you haven't read Volume 03 of the series yet, we strongly recommend starting there. It covers containers, images, Dockerfile and the essential commands you'll use a lot here.

02 • Pre-requisite: remember Docker

Before diving into Compose, it's worth revisiting the Docker concepts you'll encounter all the time here. If you read Volume 03, this will be a quick review. If it's your first time, we recommend starting there.

[DICA] Volume 03. Docker for Devs

Volume 03 of the Deploy from Zero Series covers everything you need to know before starting here: what a container is, how to write a Dockerfile, how to use Docker Hub and the essential commands. Required reading before this volume!

What you need to remember

- **Container:** uma instância em execução de uma imagem Docker. É isolado, leve e descartável.
- **Image:** o molde do container. imutável e reutilizável.
- **Dockerfile:** o arquivo que define como sua imagem é construída.
- **docker build:** constrói uma imagem a partir do Dockerfile.
- **docker run:** cria e inicia um container a partir de uma imagem.
- **Porta (-p):** mapeia uma porta do container para a sua máquina.
- **Volume:** persiste dados do container no disco da máquina.

>> If Docker is each piece of the outfit, the dress, the earrings, the bag and the shoes, then Docker Compose is the fashion app that helps you put together and save the complete look at once, with all pieces matching, without having to pick each one by hand every time you want to wear it.

Docker vs Docker Compose

Docker alone	With Docker Compose
One container at a time	All containers together
Long and repetitive commands	A single configuration file
Manual networks between containers	Automatic network between services
Manual restart of each service	<code>docker compose up</code> starts everything
Hard to share config	Version controlled file in Git

03 · What is Docker Compose used for?

Great, you already know what it is. But where will you find Docker Compose in your daily work? Let me show you the most common use cases.

Local development environment

Want to run your entire application on your machine with a single command? With Compose you define all the services, API, database, cache, queue, in one file and bring everything up together. Your colleague clones the repository, runs **`docker compose up`** and in seconds has the exact same environment as you.

Service integration

In Compose, each service can talk to another using just the service name as the hostname. The API calls the database by the name **`db`**, the authentication service calls Redis by the name **`redis`**. No need to know IPs, no extra configuration.

>> Think of your dinner table. Each chair has its place, each plate has its space, and everyone knows where to sit. Compose organizes your containers like that: each one in its place, all communicating properly, no one getting lost.

Test environments and CI/CD

CI pipelines use Docker Compose to bring up the database and dependent services before running the tests. When tests finish, everything is torn down and discarded. Clean, reproducible and without side effects.

Rapid prototyping

Want to test a new technology? MongoDB, RabbitMQ, Elasticsearch? Write a Compose file with that service, experiment and discard. Without installing anything on your real machine.

[STAT] Compose in the market

Docker Compose is standard in practically every modern development team. Knowing how to read, write and debug a docker-compose.yml is an essential skill for backend, DevOps and cloud positions. It's one of the first files you'll encounter when joining a new project.

04 · Installing Docker Compose

The good news is that if you already have Docker installed, you probably already have Docker Compose too! Starting with Docker Desktop on Windows and Mac, and recent versions of Docker Engine on Linux, Compose already comes built in as a plugin.

Windows and Mac. Docker Desktop

Docker Desktop already includes Compose. No extra installation needed. To confirm, open the terminal and run:

```
Terminal
docker compose version
# Esperado: Docker Compose version v2.x.x
```

Linux. verify and install

On Linux, Compose may need to be installed separately if you use the pure Docker Engine:

```
Terminal Linux
# Verificar se já está instalado
docker compose version
# Se não estiver, instalar como plugin
sudo apt-get update
sudo apt-get install docker-compose-plugin -y
# Confirmar instalação
docker compose version
```

[!] docker-compose vs docker compose

There is the legacy docker-compose command, which is the old V1 version, and the current docker compose without a hyphen, which is the recommended V2 version. V2 is recommended today. In this book we always use docker compose without a hyphen.

Basic file structure

The docker-compose.yml file lives at the root of your project and follows this structure:

```
docker-compose.yml. estrutura base
version: '3.8'           # versao do formato Compose
services:                # lista de containers
  nome-do-servico:
    image: nome-da-imagem
    ports:
      - 'host:container'
    environment:
      - VARIAVEL=valor
    volumes:
      - ./local:container
    depends_on:
      - outro-servico
volumes:                 # volumes nomeados
  nome-do-volume:
networks:                # redes customizadas opcional
  nome-da-rede:
```

05 · Essential Concepts

Docker Compose has its own vocabulary. Let's align on the terms you'll see in every yaml file and in every architecture discussion with the team.

Services

Definição: Each container defined in docker-compose.yml is a service. Each service has a name, image or build, ports, variables and dependencies.

>> Each band member has their instrument, their microphone and their place on stage. Compose defines each one's place before the show starts.

Networks

Definição: Compose automatically creates an internal network where all services can see each other by name. You can also create custom networks to isolate groups of services.

>> It's the event WhatsApp group. Everyone in the group can communicate. Whoever is not in the group doesn't receive the messages.

Volumes

Definição: Mechanism to persist data beyond the container lifecycle. It can be a named volume managed by Docker, or a bind mount which is a folder on your machine.

>> The eternal bag you take to every event. Whatever you store in it stays there even if the container is recreated from scratch.

depends_on

Definição: Defines the startup order of services. A service with depends_on only starts after the listed services are already running.

>> It's like the order of arrival to set up the stage: first the sound technician, then the lighting, last the musicians. Compose respects this order.

Environment (Variables)

Definição: Way to pass dynamic configurations to the container such as passwords, URLs, ports and tokens. Never write secrets directly in the Compose file!

>> The personalized script you hand to each event vendor. the client's name, the address, the time. Each one gets the information they need.

[!] depends_on does not wait for the service to be ready

The depends_on guarantees the container was started, but not that it's actually ready to receive connections. To wait for the database to truly be available, use healthchecks. You'll see this applied in Exercise 3!

Profiles

Definição: Allows marking optional services that only start when you ask for them. Great for debug tools, monitoring services or test dependencies.

>> The photographer who only comes when you book the premium package. At the basic party they don't show up, but when you want the complete record, they're there.

[DICA] Quick summary before practicing

Services are the containers. Networks are the communication networks. Volumes persist the data. `depends_on` defines the startup order. Environment passes the configurations. And Profiles activate optional services. With this you can already read any real `docker-compose.yml`!

Exercício 1 — Starting your first Compose INICIANTE

Objective: Create and run your first `docker-compose.yml` with a simple service.

Estimated time: 15 minutes

What you will learn

- How to write a **`docker-compose.yml`** from scratch
- How to use **`docker compose up`** and **`docker compose down`**
- How to view logs with **`docker compose logs`**
- How to check service status

Step by step

1. Create a folder and enter it:

```
mkdir meu-primeiro-compose  
cd meu-primeiro-compose
```

2. Create the **docker-compose.yml** file:

```
docker-compose.yml
version: '3.8'
services:
  web:
    image: nginx:alpine
    ports:
      - '8080:80'
    volumes:
      - ./html:/usr/share/nginx/html
```

Terminal

```
# Crie a pasta html e um arquivo index
mkdir html
echo "<h1>Meu primeiro Compose!</h1>" > html/index.html
```

3. Suba o serviço:

```
docker compose up -d
# Verificar que está rodando
docker compose ps
```

Access **http://localhost:8080** and see your page appear.

4. Explore the essential Compose commands:

```
docker compose logs          # ver logs de todos os serviços
docker compose logs -f web   # seguir logs do serviço web
docker compose ps            # status dos serviços
docker compose stop          # para sem remover
docker compose down          # para e remove containers e redes
docker compose down -v       # para tudo e remove os volumes
```

[OK] Expected result

O serviço Nginx subiu, você acessou pelo browser e praticou os principais comandos do Compose. Se apareceu 'Meu primeiro Compose!' na tela. missão cumprida!

[OK] Expected result. Exercise 1

O serviço Nginx subiu, você acessou pelo browser e praticou os principais comandos do Compose. Se apareceu 'Meu primeiro Compose!' na tela. missão cumprida!

Exercício 2 — App with Database

INTERMEDIARIO

Objective: Orchestrate a Python API connected to a PostgreSQL database using Docker Compose.

Estimated time: 25 to 35 minutes

What you will learn

- How to connect two services through Compose's internal network
- How to use **depends_on** to guarantee startup order
- How to use volumes to persist database data
- How to pass environment variables safely

We'll create a Flask API that connects to a PostgreSQL database, all orchestrated by Compose.

Step by step

1. Create the project structure:

```
mkdir app-com-banco && cd app-com-banco
mkdir api
```

2. Crie **api/app.py**:

api/app.py

```
# api/app.py
from flask import Flask, jsonify
import psycopg2, os
app = Flask(__name__)
@app.route('/')
def status():
    try:
        conn = psycopg2.connect(
            host=os.getenv('DB_HOST', 'db'),
            database=os.getenv('DB_NAME', 'appdb'),
            user=os.getenv('DB_USER', 'postgres'),
            password=os.getenv('DB_PASS', 'senha123')
        )
        conn.close()
        return jsonify({'status': 'conectada ao banco!'})
    except Exception as e:
        return jsonify({'erro': str(e)}), 500
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

3. Crie **api/requirements.txt** e **api/Dockerfile**:

api/requirements.txt

```
# requirements.txt
flask==3.0.0
psycopg2-binary==2.9.9
```

api/Dockerfile

```
# api/Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

4. Crie o **docker-compose.yml** na raiz:

```
docker-compose.yml
version: '3.8'
services:
  api:
    build: ./api
    ports:
      - '5000:5000'
    environment:
      - DB_HOST=db
      - DB_NAME=appdb
      - DB_USER=postgres
      - DB_PASS=senha123
    depends_on:
      - db
  db:
    image: postgres:15-alpine
    environment:
      - POSTGRES_DB=appdb
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=senha123
    volumes:
      - dados-db:/var/lib/postgresql/data
volumes:
  dados-db:
```

5. Suba tudo:

```
docker compose up -d
# Aguarde o banco inicializar e teste
curl http://localhost:5000
# Esperado: {status: conectada ao banco!}
```

>> The API is the waitress and the database is the kitchen. The waitress only starts serving when the kitchen is already running, and that's what `depends_on` guarantees. The name 'db' in the API code is the internal kitchen address, which only works because both are on the same event network.

[OK] Expected result

The API starts, connects to PostgreSQL by the name 'db' and returns the connection status. Database data persists in the volume even if you run `docker compose down` and up again.

[OK] Expected result. Exercise 2

The API starts, connects to PostgreSQL by the name 'db' and returns the connection status. Database data persists in the volume even if you run docker compose down and up again.

Exercício 3 — Complete Development Environment

DESAFIO

Objective: Set up a complete development environment with API, database, Redis cache and admin panel.

Estimated time: 40 to 50 minutes

[DICA] Spoiler of the next volume!

In this exercise you'll use environment variables via .env file, healthchecks and profiles. These are advanced Compose features that most devs only learn in practice. This environment serves as a real foundation for any project.

What you will learn

- How to use **.env file** to manage variables
- How to configure **healthchecks** to ensure healthy services
- How to use **profiles** for optional services
- How to add a **pgAdmin** panel to visualize the database

Step by step

1. Crie o arquivo **.env** na raiz do projeto (nunca suba esse arquivo pro Git!):

```
.env
# .env
POSTGRES_DB=appdb
POSTGRES_USER=postgres
POSTGRES_PASSWORD=senha_super_segura
REDIS_PASSWORD=redis_senha
PGADMIN_EMAIL=admin@deployada.com
PGADMIN_PASSWORD=admin123
```

2. Create the complete **docker-compose.yml**:

docker-compose.yml

```
version: '3.8'
services:
  api:
    build: ./api
    ports:
      - '5000:5000'
    env_file: .env
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
  db:
    image: postgres:15-alpine
    env_file: .env
    volumes:
      - dados-db:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U $POSTGRES_USER']
      interval: 5s
      timeout: 5s
      retries: 5
  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}
  pgadmin:
    image: dpage/pgadmin4
    profiles: ['tools']
    ports:
      - '5050:80'
    environment:
      - PGADMIN_DEFAULT_EMAIL=${PGADMIN_EMAIL}
      - PGADMIN_DEFAULT_PASSWORD=${PGADMIN_PASSWORD}
    depends_on:
      - db
volumes:
  dados-db:
```

3. Start the main environment and then the admin panel:

```
# Ambiente principal (sem o pgAdmin)
docker compose up -d
# Ativar o pgAdmin com profile tools
docker compose --profile tools up -d
# Acessar o pgAdmin em http://localhost:5050
```

4. Test the healthcheck and see the startup order:

```
# Acompanhar em tempo real
docker compose logs -f
# Ver status de saúde dos containers
docker compose ps
```

>> You organized the complete event: the waitress (API), the kitchen (database), the bar (Redis cache) and even a professional photographer (pgAdmin) who only appears when you activate the 'tools' profile. The .env file is the sealed envelope with the event's secrets that only the coordinator can open.

[OK] Expected result

The environment starts with healthcheck ensuring the database is ready before the API connects. Redis is available for cache. pgAdmin only starts with the tools profile active. You have a professional and reproducible development environment!

Common errors and how to fix them

- **port is already allocated:** another application is using the same port. Change the left side of the mapping, for example '5001:5000'.
- **connection refused to database:** depends_on does not guarantee PostgreSQL is ready. Use healthcheck with condition service_healthy.
- **environment variable not found:** check that the .env file is in the same folder as docker-compose.yml and that the variable name is correct.
- **volume with old data:** if the database doesn't initialize with the new config, run docker compose down with the v flag to clean the volume and recreate.

09 • Command Cheat Sheet

Cola essa página nos favoritos. São os comandos do Docker Compose que você vai usar toda semana:

Start and Stop	
<code>docker compose up</code>	Starts all services in the foreground
<code>docker compose up -d</code>	Starts all services in background
<code>docker compose up servico</code>	Starts only a specific service
<code>docker compose down</code>	Stops and removes containers and networks
<code>docker compose down -v</code>	Stops everything and removes all volumes
<code>docker compose stop</code>	Stops containers without removing
<code>docker compose start</code>	Restarts stopped containers
<code>docker compose restart</code>	Restarts all services
Monitor and Inspect	
<code>docker compose ps</code>	Lists status of all services
<code>docker compose logs</code>	Shows logs of all services
<code>docker compose logs -f api</code>	Follows logs of the api service
<code>docker compose top</code>	Lists processes running in containers
<code>docker compose stats</code>	Monitors CPU and memory usage
Build and Images	
<code>docker compose build</code>	Builds all images with build
<code>docker compose build api</code>	Builds only the api service image
<code>docker compose pull</code>	Downloads the latest images
<code>docker compose push</code>	Pushes images to the registry

Execution and Debug	
<code>docker compose exec api bash</code>	Opens terminal in the api service
<code>docker compose run api python -c ...</code>	Executes one-off command in the service
<code>docker compose config</code>	Validates and shows the final compose
<code>docker compose --profile tools up</code>	Starts services of a specific profile

10 • Next Steps

Congratulations on making it here! You can now orchestrate complete environments with Docker Compose. The series continues with topics that will use everything you learned here:

Vol. 05. APIs REST com Python

Aprenda a construir APIs completas com Flask ou FastAPI, com boas práticas de REST, autenticação e documentação. Tudo dockerizado e com Compose, claro.

Vol. 06. AWS na Prática

With Compose mastered locally, we go to the cloud. ECS, ECR, S3 and Lambda are the next destinations to take your containers to production on AWS.

Vol. 07. Mensageria: SQS & SNS

Learn to build asynchronous systems with message queues. Compose will orchestrate your API, worker and broker all together.

Recursos para continuar estudando

- **Documentação oficial:** docs.docker.com/compose. referência completa e atualizada
- **Awesome Compose:** github.com/docker/awesome-compose. exemplos reais de Compose
- **Play with Docker:** labs.play-with-docker.com. pratique no browser sem instalar nada
- **Livro:** Docker Deep Dive de Nigel Poulton. capítulo dedicado ao Compose

What you mastered in this volume

- What Docker Compose is and why it's essential in the market
- How to structure a docker-compose.yml file from scratch
- How to connect multiple services with internal networks
- How to use depends_on, healthchecks and volumes
- How to manage variables with .env file
- How to use profiles for optional services

You made it here. That already puts you ahead of a lot of people.

Docker Compose is not just a tool. It's the difference between an environment that only works on your machine and one that works anywhere in the world.

Did you enjoy this mini book?

Share it with that dev friend who's just starting their journey!

Tag me on Instagram **@deployada** with your progress.

I love watching you all grow!

Instagram	LinkedIn	Newsletter
@deployada	linkedin.com/in/mgcarvalho	deployada

Continue sua jornada! >>

Vol. 04 • Docker Compose. orquestrando containers

Vol. 05 • APIs REST com Python

Vol. 06 • AWS na prática: S3, EC2 e Lambda

Vol. 07 • Mensageria: SQS & SNS

Vol. 08 • Observabilidade: Datadog & Splunk

"Every great dev ran their first container once."

— @deployada

@deployada

Instagram • LinkedIn • Newsletter